JINA EMBEDDINGS 2: 8192-Token General-Purpose Text Embeddings for Long Documents

Michael Günther, Jackmin Ong, Isabelle Mohr, Alaeddine Abdessalem, Tanguy Abel, Mohammad Kalim Akram, Susana Guzman, Georgios Mastrapas, Saba Sturua, Bo Wang, Maximilian Werk, Nan Wang and **Han Xiao**
Jina AI GmbH, Ohlauer Str. 43, 10999 Berlin, Germany
{michael.guenther, jackmin.ong, isabelle.mohr alaeddine.abdessalem, tanguy.abel, kalim.akram, susana.guzman, georgios.mastrapas, saba.sturua, bo.wang, maximilian.werk, nan.wang, han.xiao}@jina.ai

arXiv:2310.19923v4 [cs.CL] 4 Feb 2024

Abstract

Text embedding models have emerged as powerful tools for transforming sentences into fixed-sized feature vectors that encapsulate semantic information. While these models are essential for tasks like information retrieval, semantic clustering, and text re-ranking, most existing open-source models, especially those built on architectures like BERT, struggle to represent lengthy documents and often resort to truncation. One common approach to mitigate this challenge involves splitting documents into smaller paragraphs for embedding. However, this strategy results in a much larger set of vectors, consequently leading to increased memory consumption and computationally intensive vector searches with elevated latency.

To address these challenges, we introduce `Jina Embeddings v2`, an open-source text embedding model[1] capable of accommodating up to 8192 tokens. This model is designed to transcend the conventional 512-token limit and adeptly process long documents. `Jina Embeddings v2` not only achieves state-of-the-art performance on a range of embedding-related tasks in the MTEB benchmark but also matches the performance of OpenAI's proprietary `text-embedding-ada-002` model. Additionally, our experiments indicate that an extended context can enhance performance in tasks such as NarrativeQA.

1 Introduction

Using neural networks to encode text and images into embedding representations has become a standard practice for analyzing and processing vast amounts of unstructured data. In natural language processing, sentence embedding models [Reimers and Gurevych, 2019] transform the semantics of phrases, sentences, and paragraphs into points within a continuous vector space. These transformed data points can subsequently be used for a myriad of downstream applications, such as information retrieval, as well as clustering and classification tasks.

Despite the numerous applications of embedding models, a prevailing challenge faced by many models is the limitation on the maximum sequence lengths of text that can be encoded into a single embedding. To circumvent this, practitioners often segment documents into smaller chunks prior to encoding. This tactic, unfortunately, results in fragmented semantic meanings, causing the embeddings to misrepresent the entirety of paragraphs. Furthermore, this method yields a plethora of vectors, culminating in heightened memory usage, increased computational demands during vector searches, and extended latencies. The dilemma is exacerbated when embedding vectors are stored in database systems that construct memory-intensive index structures.

The root of these text length restrictions can be traced back to the BERT architecture, which underpins most of the current open-source models. The authors of [Press et al., 2022] demonstrated that these models struggle to accurately represent long documents. They introduced an alternative positional embedding method named ALiBi, enabling efficient training of models to encode long text sequences. Regrettably, up until this point, the approach was exclusively employed for generative language models, neglecting its potential for open-source encoder language models aimed at crafting document embeddings. This research bridges that gap by incorporating ALiBi bidirectionally into the BERT framework, rendering it apt for encoding tasks. As a result, it empowers users to utilize it for downstream operations on texts spanning up to 8192 tokens. Moreover, we

[1] Base model (0.27G): https://huggingface.co/jinaai/jina-embeddings-v2-base-en
Small model (0.07G): https://huggingface.co/jinaai/jina-embeddings-v2-small-en
API: https://jina.ai/embeddings

fine-tuned this enhanced BERT model, harnessing hundreds of millions of text samples to encode texts into singular embedding representations. Our model's resultant embeddings outshine those of the `Jina Embeddings v1` model suite [Günther et al., 2023] in the MTEB benchmark and rival the prowess of state-of-the-art models like E5 [Wang et al., 2022]. We also found that large context lengths can amplify the efficacy of numerous downstream tasks tied to embeddings. Given that the majority of available embedding evaluation datasets comprise mainly brief text passages, we have curated datasets encompassing long text values to better evaluate embeddings. These datasets, alongside our models, are made accessible via our Hugging Face repository[2].

This paper is structured as follows: We begin with an overview of related work in Section 2. This is followed by an outline of the training paradigm in Section 3, a description of the backbone model and its pre-training in Section 4, and a detailed walkthrough of the fine-tuning process for embeddings generation in Section 5. We culminate with an exhaustive evaluation in Section 6 and conclusions in Section 7.

2 Related Work

Embedding training has undergone significant evolution, transitioning from foundational techniques such as Latent Semantic Indexing (LSA) [Deerwester et al., 1990] and Latent Dirichlet Allocation (LDA) [Blei et al., 2001] to the sophisticated prowess of pre-trained models like Sentence-BERT [Reimers and Gurevych, 2019]. A notable shift in recent advancements is the emphasis on unsupervised contrastive learning, as showcased by works like [Gao et al., 2022, Wang et al., 2022]. Pioneering models like Condenser [Gao and Callan, 2021] and RetroMAE [Xiao et al., 2022] have brought forth specialized architectures and pre-training methods explicitly designed for dense encoding and retrieval.

The E5 [Wang et al., 2022], `Jina Embeddings v1` [Günther et al., 2023], and GTE [Li et al., 2023] collections of embedding models represent another leap forward. These models propose a holistic framework tailored for effective training across a myriad of tasks. This framework adopts a multi-stage contrastive training approach. An initial phase focuses on training using a vast collection of weak pairs sourced from public data, enhancing the model's domain generalization. Following this, a supervised fine-tuning stage employs a curated set of annotated text triples, representing diverse tasks. Together, these sequential stages yield state-of-the-art outcomes on the MTEB benchmark.

Yet, despite such advancements, a glaring limitation persists: the 512-token constraint on input sequences, stemming from foundational models like BERT. This cap is insufficient for encoding lengthy documents, often exceeding a page. ALiBi [Press et al., 2022] emerges as a promising solution, presenting a technique that sidesteps conventional positional embeddings and facilitates training on sequences exceeding 2048 tokens. Notably, its typical application is centered around generative models, which inherently adopt a unidirectional bias, rendering it less suitable for embedding tasks.

Effective evaluation remains paramount for embedding models, ensuring they meet the diverse demands of real-world applications. The BEIR benchmark [Thakur et al., 2021] stands out, offering evaluations across a set of retrieval tasks and settings. Similarly, the MTEB benchmark [Muennighoff et al., 2023] highlights the extensive applicability of text embeddings, spanning a variety of tasks and languages. However, a notable gap in both benchmarks is their limited focus on encoding long documents — a critical aspect for comprehensive embedding evaluation.

3 Training Paradigm Overview

The training paradigm for `Jina Embeddings v2` is divided into three stages:

I **Pre-training a Modified BERT:** For the backbone language model, we propose a modified BERT model capable of encoding documents with up to 8192 tokens. This model is trained from scratch on a full-text corpus using a masked language modeling objective.

II **Fine-tuning with Text Pairs:** To encode a text passage into a single vector representation, the model is fine-tuned on text pairs.

III **Fine-tuning with Hard Negatives:** The model is further fine-tuned using text pairs complemented with hard negatives. This stage is crucial for enabling the model to better

[2] https://huggingface.co/jinaai

Model	Layers	Hidden	Params
Jina BERT Small	4	512	33M
Jina BERT Base	12	768	137M
Jina BERT Large	24	1024	455M

Table 1: Architecture specifications for the Jina BERT models of varying sizes. The number of attention heads is selected to ensure a consistent head dimension of 64.

distinguish between relevant passages and related, but irrelevant text passages.

While both stages II and III are geared towards training the models for embedding tasks, the latter is especially critical for improving the model's performance in retrieval and classification tasks (refer to Section 6.2).

4 Pre-training a Modified BERT

For the backbone language model, we introduce a novel transformer based on BERT [Devlin et al., 2019] with several modifications to enhance its ability to encode extended text sequences and to generally bolster its language modeling capabilities. For the training process, we largely adopt the approach described in [Liu et al., 2019a], incorporating additional performance optimizations.

4.1 Model Architecture

Attention with Linear Biases: For the self-attention mechanism within the attention blocks, we adopt the Attention with Linear Biases (ALiBi) approach [Press et al., 2022]. ALiBi forgoes the use of positional embeddings. Instead, it encodes positional information directly within the self-attention layer by introducing a constant bias term to the attention score matrix of each layer, ensuring that proximate tokens demonstrate stronger mutual attention. While the original implementation was designed for causal language modeling and featured biases solely in the causal direction, such an approach is not compatible with the bidirectional self-attention inherent in our encoder model. For our purposes, we employ the symmetric encoder variant where attention biases are mirrored to ensure consistency in both directions[3]. Figure 1 depicts the computation of attention scores within the multi-head attention heads. Each head's scaling value, m_i, out of the total n heads, is derived using Equation (1).

$$m_i = \begin{cases} b^{2i} & i < a \\ b^{1+2(i-a)} & i \geq a \end{cases}$$
$$a = 2^{\lfloor \log_2 n \rfloor} \quad b = 2^{\frac{-8}{2^{\lceil \log_2 n \rceil}}} \tag{1}$$

Gated Linear Units: For the feedforward sublayers within the attention blocks, we adopt Gated Linear Units (GLU), originally introduced in [Dauphin et al., 2016]. They've demonstrated performance enhancements when incorporated into transformers [Shazeer, 2020]. For the small and base models, we employ the GEGLU variant, which leverages the GELU activation function for the GLU. Conversely, for the large model, we utilize the ReGLU variant with the ReLU activation function. This choice was driven by our observation that training the large model with GEGLU, despite its promising initial MLM accuracy, was unstable.

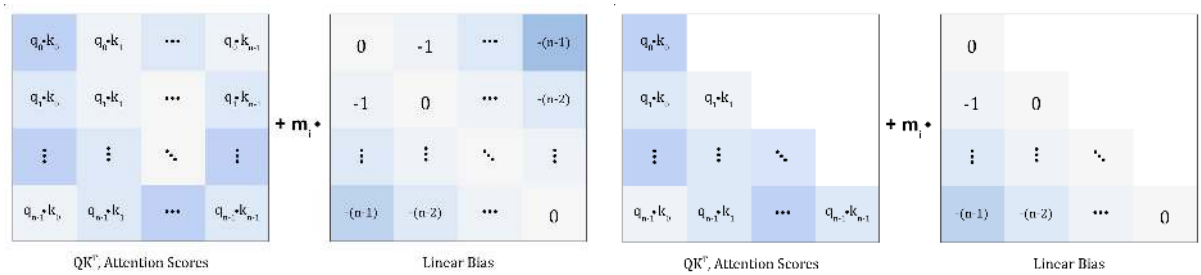
Layer Normalization: Regarding Layer Normalization [**?**], we align with the post-layer normalization approach from [Vaswani et al., 2017] in our attention blocks. Preliminary tests with pre-layer normalization, as mentioned in [Shoeybi et al., 2019] and [Nguyen and Salazar, 2019], didn't enhance training stability or performance. Consequently, we opted not to integrate it into our model.

4.2 Training Data

For the pre-training phase, we leverage the English "Colossal, Cleaned, Common Crawl (C4)" dataset [4], encompassing approximately 365 million text documents harvested from the web, summing to around 170 billion tokens. As delineated in [Raffel et al., 2020], the C4 dataset is a refined iteration of Common Crawl, utilizing heuristics for cleanup and language recognition, retaining solely English content. As a result, our models are monolingual and tailored exclusively for English texts. The purification process also encompasses the removal of webpages hosting inappropriate content. We reserve 1% of the dataset for evaluating validation loss and the accuracy of the masked language modeling (MLM) task.

[3] https://github.com/ofirpress/attention_with_linear_biases/issues/5

[4] https://huggingface.co/datasets/c4



(a) Encoder ALiBi (b) Causal ALiBi

Figure 1: With ALiBi attention, a linear bias is incorporated into each attention score preceding the softmax operation. Each attention head employs a distinct constant scalar, m, which diversifies its computation. Our model adopts the encoder variant where all tokens mutually attend during calculation, contrasting the causal variant originally designed for language modeling. In the latter, a causal mask confines tokens to attend solely to preceding tokens in the sequence.

4.3 Training Algorithm

Our model's pre-training revolves around the masked language modeling objective, excluding the next sentence prediction (NSP) task due to its perceived limited contribution to downstream task performance [Liu et al., 2019a]. We mask 30% of the input tokens randomly, employing whole word masking [Devlin et al., 2019], and condition the models to infer these masked tokens. Of these masked tokens, 80% are substituted with the `[MASK]` token, 10% with a random token, and the remaining 10% stay unaltered.

The masked tokens are predicted by a decoder $f : \mathbb{R}^d \to \mathbb{R}^{|V|}$, which takes the output token embedding $\boldsymbol{e_i} \in \mathbb{R}^d$ of a masked token and predicts a probability for each token in the vocabulary. The loss L_{MLM} is computed by evaluating the cross entropy between the predicted probabilities and the actual masked tokens, as described in Equation (2). Here, $I : \{1, \ldots, n\} \to |V|$ denotes the function that maps each of the n masked tokens to its respective index in the vocabulary:

$$\mathcal{L}_{\text{MLM}}(t) := \sum_{k=1}^{n} \ln f(\boldsymbol{e_i})_{I(k)} \qquad (2)$$

Given our model's reliance on ALiBi attention [Press et al., 2022], training position embeddings becomes unnecessary. This allows us to pre-train more efficiently on shorter sequences and adapt to longer sequences in subsequent tasks. Throughout our pre-training, we operate on sequences capped at 512 tokens in length. Diverging from the methods in [Devlin et al., 2019] and [Liu et al., 2019a], our sequences originate from individual documents without any multi-document packing. Furthermore, we refrain from sampling multiple sequences from a singular document. For each document, we exclusively consider its initial 512 tokens, truncating any excess. Given our consistent global batch size of 4096, each batch, due to its varying sequence length, contains a unique number of masked tokens when calculating loss.

Optimizer: Mirroring the optimization strategy of RoBERTa [Liu et al., 2019a], we employ the AdamW algorithm [Loshchilov and Hutter, 2017], characterized by parameters $\beta_1 = 0.9$, $\beta_2 = 0.98$, $\epsilon = 1\text{e}{-6}$, a weight decay of 0.01, dropout set at 0.1, and attention dropout also at 0.1. Our learning rate schedule is linear, starting at 0 and peaking at a rate of η post $10,000$ steps. Here, the values of η are designated as $1\text{e}{-3}$, $6\text{e}{-4}$, and $4\text{e}{-4}$ for the small, base, and large models respectively. A linear decay to zero ensues after reaching the $100,000$ steps threshold.

Mixed precision training: We resort to FP16 dynamic mixed precision [Micikevicius et al., 2018] for pre-training our models, facilitated by the DeepSpeed software package [Rasley et al., 2020]. Our preliminary tests using BF16 revealed unsatisfactory performance metrics, both in MLM accuracy and the downstream GLUE tasks.

5 Fine-Tuning for Embeddings

After pre-training the `JinaBERT` models, we further fine-tune each of the models to encode a text sequence into a single vector representation. The core idea behind our embedding approach is inspired by the Sentence-BERT [Reimers and Gurevych, 2019]. To enable a model to perform a text operation, we augment it with a mean pooling layer. This mean pooling step averages the token embed-

dings to merge their information into a single representation, without introducing additional trainable parameters. The training process for this enhanced model consists of an unsupervised phase followed by a supervised one.

5.1 Fine-tuning with Text Pairs

During the first fine-tuning stage, we train the models on a corpus of text pairs $(q, p) \in \mathbb{D}^{\text{pairs}}$, comprising a query string q and a target string p.

Training Data We utilize roughly 40 diverse data sources, akin to the data preparation outlined in the report we previously published about our inaugural embedding model suite [Günther et al., 2023]. We observed that the inclusion of title-abstract pairs from documents significantly enhances performance on clustering tasks. As detailed in [Günther et al., 2023], we implement consistency filtering [Dai et al., 2023, Wang et al., 2022] to elevate the quality of the text pair corpus. For batch creation, we adhere to our earlier strategy: for every new batch, we randomly choose a data source and extract as many pairs as needed to fill that batch. All pairs within the data sources are pre-shuffled. Depending on the quality and quantity of the data sources, we assign different sampling rates for the pairs.

Loss Function: The goal of this fine-tuning stage is to encode text values that constitute a pair into analogous embedding representations, while encoding texts that aren't paired into distinct embeddings. To achieve this contrastive goal, we employ the InfoNCE [van den Oord et al., 2018] loss function, similar to our earlier embedding models [Günther et al., 2023]. This loss function calculates the loss value for a pair $(q, p) \sim \mathbf{B}$ within a batch $\mathbf{B} \subset \mathbb{D}^{\text{pairs}}$ as follows:

$$\mathcal{L}^{\text{pairs}}_{\text{NCE}}(\mathbf{B}) := \mathbb{E}_{(q,p)\sim\mathbf{B}}\left[-\ln \frac{e^{s(q,p)/\tau}}{\sum_{i=1}^{k} e^{s(q,p_i)/\tau}}\right] \quad (3)$$

The function evaluates the cosine similarity $s(p, q)$ between a given query q and its corresponding target p, relative to the similarity of all other targets in the batch. Given the typically symmetric nature of similarity measures, we compute the loss in both directions:

$$\mathcal{L}^{\text{pairs}}(\mathbf{B}) := \mathcal{L}^{\text{pairs}}_{\text{NCE}}(\mathbf{B}) + \mathcal{L}^{\text{pairs}}_{\overline{\text{NCE}}}(\mathbf{B}), \text{ with}$$

$$\mathcal{L}^{\text{pairs}}_{\overline{\text{NCE}}}(\mathbf{B}) := \mathbb{E}_{(q,p)\sim\mathbf{B}}\left[-\ln \frac{e^{s(p,q)/\tau}}{\sum_{i=1}^{k} e^{s(p,q_i)/\tau}}\right] \quad (4)$$

The constant temperature parameter τ influences how the loss function weighs minor differences in the similarity scores [Wang and Liu, 2021]. Empirical testing suggests that $\tau = 0.05$ is effective.

5.2 Fine-tuning with Hard Negatives

The goal of the supervised fine-tuning stage is to improve the models' ranking capabilities. This improvement is achieved by training with datasets that include additional negative examples.

Training Data We have prepared retrieval datasets, such as MSMarco [Bajaj et al., 2016] and Natural Questions (NQ) [Kwiatkowski et al., 2019], in addition to multiple non-retrieval datasets like the Natural Language Inference (NLI) dataset [Bowman et al., 2015]. These datasets encompass a collection of queries with annotated relevant passages and several negative examples, consistent with earlier work [Wang et al., 2022]. Each training batch B, structured as $(q, p, n_1, \ldots, n_{15})$, includes one positive and 15 negative instances. For retrieval datasets, hard negatives are discerned by identifying passages deemed similar by retrieval models. This approach instructs the model to prioritize relevant documents over those that are merely semantically related. For non-retrieval datasets, negatives are selected randomly, since drawing a clear line between positives and hard negatives isn't feasible. This is because, unlike relevancy, it's challenging to make a binary determination regarding the similarity or dissimilarity of two textual values. Consequently, opting for hard negatives in such datasets seemed to diminish the models' quality. Nonetheless, it remains crucial to integrate these datasets into the stage III training to ensure continued performance on non-retrieval tasks. To ensure that hard negative passages are indeed less relevant than the annotated relevant ones, we employ a cross-encoder model to validate that their relevance score is indeed lower.

Loss Function: Our training employs a modified variant of the InfoNCE loss function, denoted as $\mathcal{L}_{\text{NCE}^+}$ and described by Equation (5). Similar to

the preceding loss function, this one is bidirectional and incorporates the additional negatives when pairing queries with passages:

$$\mathcal{L}_{\text{NCE}^+}(B) :=$$

$$\mathbb{E}_{r\sim B}\left[-\ln \frac{e^{s(q,p)/\tau}}{\sum_{i=1}^{k}\left[e^{s(q,p_i)/\tau} + \sum_{j=1}^{15} e^{s(q,n_{j,i})/\tau}\right]}\right]$$

$$+\,\mathbb{E}_{r\sim B}\left[-\ln \frac{e^{s(p,q)/\tau}}{\sum_{i=1}^{k} e^{s(p,q_i)/\tau}}\right]$$

$$\text{with } r = (q, p, n_1, \ldots, n_{15}). \tag{5}$$

5.3 Memory Optimizations

When training embedding models, having a large batch size is crucial. This is because the InfoNCE loss functions $\mathcal{L}^{\text{pairs}}$ and L_{NCE^+} compute the loss values based on the entirety of the batch. The batch size determines the number of text values each individual text value is compared against. As a result, the computed loss value might not be as expressive with smaller batches. Li et al. [2023] provided an in-depth analysis, highlighting the positive impact of larger batch sizes on the performance of the resultant embedding model. To accommodate larger batch sizes, it becomes essential to minimize the memory overhead during training. We achieved this by training our models in mixed precision [Micikevicius et al., 2018] and leveraging the deepspeed [Rasley et al., 2020] framework for further optimization. Activation checkpointing [Chen et al., 2016] was also employed to curtail memory usage. Specifically, we inserted a checkpoint after each BERT layer within our model.

6 Evaluation

To evaluate the efficacy of our approach, we initiate with a comprehensive analysis of our pre-trained backbone models, as outlined in Section 6.1. This is followed by an in-depth assessment of our embedding models in Section 6.2. Furthermore, we have conducted experiments to delve into the effects of encoding extended sequence lengths on the performance of the embeddings, presented in Section 6.2.2.

6.1 Evaluation of Jina BERT

Following previous work [Liu et al., 2019b], we evaluate our pretrained models on the GLUE benchmark [Wang et al., 2018]. General Language Understanding Evaluation (GLUE) is a collection of

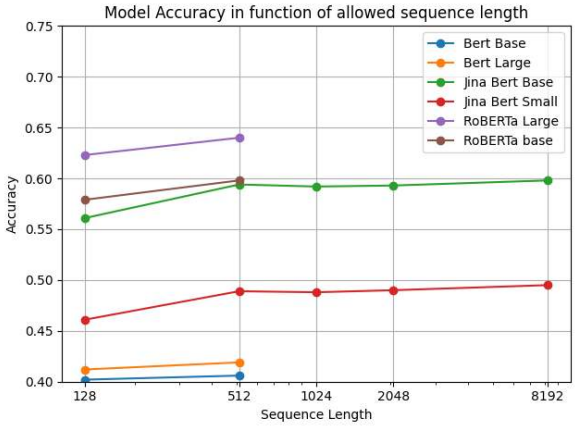


Figure 2: Variation of model MLM accuracy w.r.t. the sequence length

nine datasets for evaluating natural language understanding systems. Six tasks are framed as either single-sentence classification or sentence-pair classification tasks. The GLUE organizers provide training, development, and test data splits, as well as a submission server and leaderboard.[5] The test split does not contain labels, and the submission server allows participants to evaluate and compare their systems against the private labels of the test split.

For the Jina BERT training described in Section 4, we fine-tune the pre-trained models on the corresponding single-task training data using several hyperparameter settings and, for each task, pick the best fine-tuning hyperparameters on the development set.

Following the methodology of [Phang et al., 2018], for RTE, STS, and MRPC, we fine-tune starting from the MNLI single-task model, rather than the baseline pretrained Jina BERT models. As in the BERT paper [Devlin et al., 2019], our fine-tuning procedure relies on representing the input sequence and using the final hidden vector $C \in \mathbb{R}^H$ corresponding to the first input token ([CLS]) as the aggregate representation.

We train for 10 epochs with batch sizes $\{16, 32\}$ and learning rates $\{1\text{e}{-5}, 2\text{e}{-5}, 3\text{e}{-5}\}$. For each task, the best fine-tuned model on the development set is used for the test set.

In Table 2, we report the results of the best-performing models on the test sets after submission to the GLUE benchmark server.

Furthermore, we evaluate Jina BERT models on documents of long text sequences by computing

[5] https://gluebenchmark.com

Model	Params	MNLI	QQP	QNLI	SST-2	CoLa	STS-B	MRPC	RTE	WNLI	Average
BERT Base	110M	84.6/83.4	71.2	90.5	93.5	52.1	85.8	88.9	66.4	-	-
BERT Large	340M	86.7/85.9	72.1	92.7	94.9	60.5	86.5	89.3	70.1	-	-
RoBERTa	355M	90.8/90.2	90.2	98.9	96.7	67.8	92.2	92.3	88.2	89.0	88.5
Jina BERT Small	33M	80.1/78.9	78.9	86.0	89.6	28.8	84.8	84.1	68.8	55.5	72.9
Jina BERT Base	137M	85.7/85.4	80.7	92.2	94.5	51.4	89.5	88.4	78.7	65.1	80.7
Jina BERT Large	435M	86.6/85.9	80.9	92.5	95.0	59.6	88.2	88.5	78.5	65.1	81.6

Table 2: Evaluation of the Jina BERT models on the GLUE benchmark

the accuracy of the MLM task with varying sequence lengths. The accuracy of masked language modeling is computed on $50,000$ samples from the C4 validation set where, for each chosen sequence length, each sample document is tokenized and truncated to fit the sequence length. We compare Jina BERT to RoBERTa and BERT models in Figure 2. It essentially shows that, even though Jina BERT models were trained on a 512 sequence length, the MLM accuracy does not drop when we extrapolate to an 8192 sequence length. For other BERT and RoBERTa models, since they use absolute positional embeddings that are trained on a 512 sequence length, it's not possible to compute the MLM accuracy beyond 512. The figure demonstrates ALiBi's effectiveness in maintaining MLM performance during inference for long documents.

6.2 Evaluation of Jina Embeddings v2

To comprehensively evaluate our embedding models, we employ the Massive Text Embedding Benchmark (MTEB) [Muennighoff et al., 2023]. Our choice of MTEB is motivated by its unparalleled breadth, distinguishing it among embedding benchmarks. Rather than focusing on a single task and dataset, MTEB covers an expansive set of 8 tasks, encompassing a rich collection of 58 datasets across 112 languages. This expansive benchmark allows us to scrutinize our model's adaptability across diverse applications and languages and benchmark it against other top-performing models.

However, a limitation of the MTEB benchmark is its omission of very long texts, which are essential for evaluating our model's prowess in handling 8192 sequence lengths. Consequently, we introduce new retrieval and clustering tasks featuring extended documents, and we detail the performance of our model against its peers in Section 6.2.2.

Clustering: The goal here is to aptly group a collection of sentences or paragraphs. Within the MTEB benchmark suite, a mini-batch k-means model is employed, operating with a batch size of 32. Here, k represents the number of unique labels in the dataset. Model performance is evaluated using the $\mathcal{V}$ measure, a metric insensitive to cluster label permutations, guaranteeing that assessments are independent of label configurations.

We incorporate two new clustering tasks featuring extended documents within the MTEB clustering task subset. The inaugural task, named PatentClustering, draws from the BigPatent[6] dataset [Sharma et al., 2019], challenging the k-means model to organize patents by their respective categories. Patent documents average $6,376$ tokens, spanning a range from a brief 569 tokens to an extensive $218,434$ tokens. Our second task, titled WikiCitiesClustering, sources from the English subset of the refined Wikipedia dump [Foundation, 2022], available as a dataset on Hugging Face[7]. For this task, we curate a roster of nations from Wikidata and extract Wikipedia articles of their cities from the refined dataset. The objective is to group cities by their parent country. On average, articles consist of $2,031$ tokens, with the length varying between a succinct 21 tokens to a comprehensive $20,179$ tokens.

Retrieval: This task entails a dataset comprising a corpus, a set of queries, and associated mappings connecting each query to pertinent corpus documents. The mission is to discern relevant documents for a specific query. Both queries and corpus documents undergo encoding, after which their similarity scores are derived using cosine similarity. Subsequently, metrics like nDCG@10 (which serves as the primary metric), MRR@k, MAP@k, precision@k, and recall@k are computed for diverse k values. This task is inspired by datasets and evaluation methods presented by BEIR [Thakur et al., 2021].

To expand the scope of the MTEB, we introduce a new retrieval task named NarrativeQA, de-

[6] https://huggingface.co/datasets/big_patent
[7] https://huggingface.co/datasets/wikipedia

rived from the narrativeqa[8] dataset. This dataset boasts realistic QA instances, curated from literature (encompassing both fiction and non-fiction) and film scripts. The corpus averages 74,843 tokens per document, with the lengthiest document tallying up to 454,746 tokens, and the most concise one comprising 4,550 tokens.

We further evaluated Jina Embeddings v2 using a novel benchmark, referred to as LoCo [9]. The LoCo dataset consists of five retrieval tasks derived from publicly available datasets. The selection process for these tasks was guided by several criteria, notably the length of the documents, with a preference towards longer texts, in addition to a manual review to verify that the tasks require a thorough understanding of the entire document. The results of our models on the LoCo dataset are provided in Table 11.

6.2.1 Results on MTEB

The evaluation of embedding models within the MTEB benchmark, as illustrated in Table 3, reveals significant contrasts between Jina's text embedding models, namely jina-small-v2 and jina-base-v2, and other contemporary models. These differences are especially pronounced in tasks showing marked performance disparities, such as Classification (CF) and Retrieval (RT).

In Classification (CF), the jina-base-v2 model, equipped with 137 million parameters, emerges as a leading performer. It records superior scores, outpacing most competing models, underscoring its efficacy in text classification. Conversely, the jina-small-v2 model, equipped with a modest 33 million parameters, trails behind some other models in this task. This underscores the pivotal role model size plays in certain downstream tasks, with more extensive architectures yielding potential benefits.

For the Retrieval (RT) task, jina-small-v2 showcases formidable performance, signaling its adeptness for information retrieval. It ranks amidst top-tier models, indicating its prowess in retrieval-centric tasks. Similarly, jina-base-v2 excels, registering a slightly superior score, reaffirming its formidable retrieval aptitude. Both models underscore their credibility in tasks necessitating adept information retrieval. Given that models all-MiniLM-L6-v2 and all-mpnet-base-v2 omit the second-stage finetuning which jina-small-v2 and jina-base-v2 undergo, it's foreseeable that our models would excel in these tasks.

In conclusion, both the base and small text embedding models display commendable performance within the MTEB benchmark. Their standout performance, relative to other models in tasks like Classification and Retrieval, suggests model size's influential role in specific text processing endeavors. Both models reaffirm their potency in retrieval, marking them as pivotal tools for a plethora of natural language processing tasks.

6.2.2 Impact of Maximum Sequence Length

As delineated in Section 6.1, the pre-training generalizes across extended sequence lengths. Consequently, the MLM accuracy for long sequences, spanning up to 8192 tokens, mirrors that of shorter sequences, despite the exclusive training on abbreviated text sequences. During finetuning, our models train solely on texts not exceeding 512 tokens, yet they cater to texts reaching 8192 tokens for the MTEB evaluation detailed in Section 6.2.

To discern how sequence length impacts the accuracy of downstream tasks, we executed long document clustering and retrieval tasks, modulating the tokenizer's maximum sequence length. This allows us to gauge the models' performance on variable sequence lengths through truncation. Since a majority of the extant tasks in the MTEB feature documents under 512 tokens, we resort to our three novel datasets elucidated in Section 6.2, accessible on Hugging Face. Furthermore, we employ the SciFact dataset [Wadden et al., 2020], given its substantial count of texts exceeding 512 tokens.

Figure 3 depicts the nDCG@10 retrieval and the $\mathcal{V}$ measure scores for the jina-base-v2 alongside four other renowned embedding models. Given that only jina-base-v2 and OpenAI's text-embedding-ada-002 support an 8K sequence length, results reported for an 8191 sequence length for other models are truncated to their intrinsic maximum, typically 512. Generally, Figure 3 suggests that elongated sequence lengths contribute to enhanced outcomes. This assertion is particularly true for the NarrativeQA task, where extending the sequence length substantially bolsters performance. Due to the inherent nature of the dataset, models limited to the text's commencement frequently underperform.

On the BigPatent clustering task, larger sequence

[8] https://huggingface.co/datasets/narrativeqa

[9] https://hazyresearch.stanford.edu/blog/2024-01-11-m2-bert-retrieval

Model	Params	CF	CL	PC	RR	RT	STS	SM	Average
`text-embedding-ada-002`	unknown	70.93	45.90	84.89	56.32	49.25	80.97	30.80	60.99
e5-base-v2	110M	73.84	43.80	85.73	55.91	50.29	81.05	30.28	61.50
all-MiniLM-L6-v2	23M	63.05	42.35	82.37	58.04	41.95	78.90	30.81	56.26
all-mpnet-base-v2	110M	65.07	43.69	83.04	59.36	43.81	80.28	27.49	57.78
`jina-small-v2`	33M	68.82	40.08	84.44	55.09	45.64	80.00	30.56	58.12
`jina-base-v2`	137M	73.45	41.74	85.38	56.99	48.45	80.70	31.60	60.37

CF: Classification Accuracy [%] CL: Clustering $\mathcal{V}$ measure[%] PC: Pair Classification Average Precision [%]
RR: Reranking MAP [%] RT: Retrieval nDCG@10 STS: Sentence Similarity Spearman Correlation [%]
SM: Summarization Spearman Correlation [%]

Table 3: Evaluation of the `Jina Embeddings v2` models on the MTEB benchmark

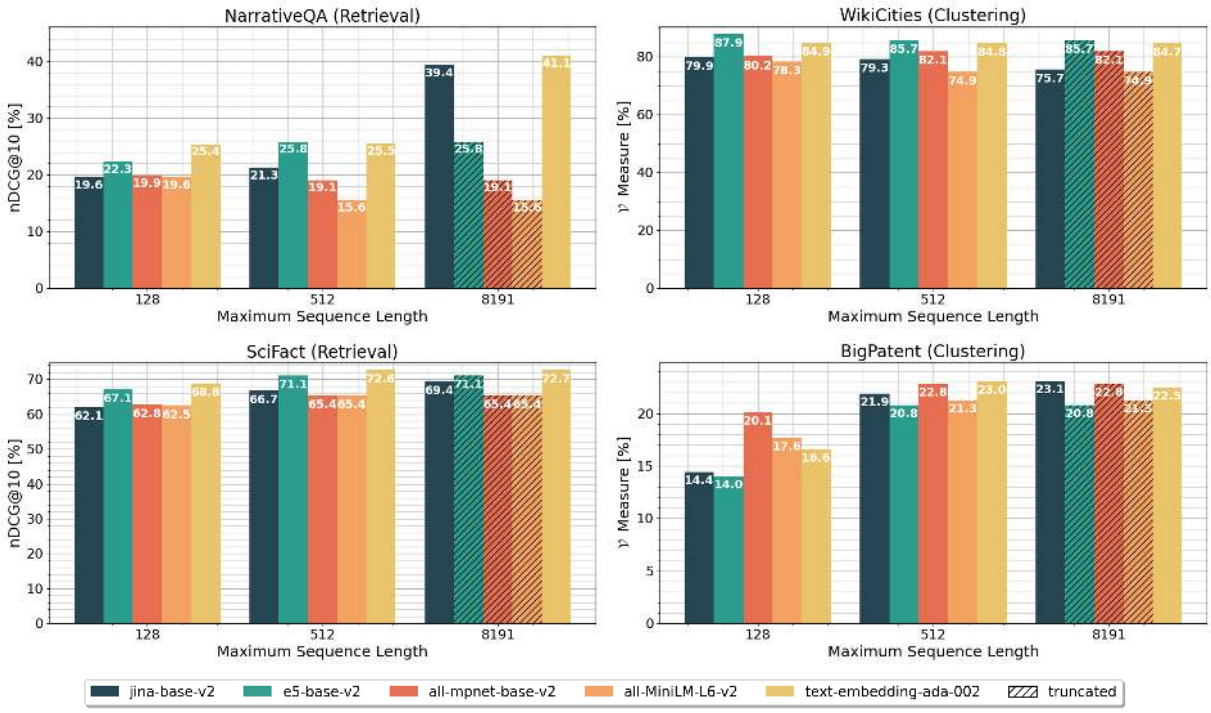


Figure 3: Evaluation w.r.t. maximum sequence length. For `e5-base-v2`, we abstained from employing specific prefixes like `query: `, which might result in varied evaluation outcomes. Note, `text-embedding-ada-002` caps its context length at 8191 tokens, not 8192.

lengths also result in better performance. However, on the WikiCities clustering task, longer sequence lengths seem to slightly diminish the models' performance in most instances. This suggests that an increase in sequence length doesn't always yield better outcomes. One explanation for this observation is that the initial paragraph of a Wikipedia article about a city typically mentions the country the city is in. Information towards the middle and end of the articles is often less pertinent for identifying the country and might alter the attributes that influence the clustering of the city embeddings.

7 Conclusion

We have introduced `Jina Embeddings v2`, a novel embedding model based on a modified BERT architecture. This model eschews positional embeddings and instead employs bi-directional ALiBi slopes to capture positional information. By training a series of embedding models with this innovative architecture on the Web document corpus C4 and subsequently fine-tuning them, we have enabled the encoding of the semantics of both short and long textual values into meaningful vector representations. This effort has produced a new suite of open-source embedding models capable of encoding texts containing up to 8192 tokens. These embeddings signify a 16x increase in the maximum

sequence length compared to leading open-source embedding models. Additionally, our model suite exhibits competitive performance on the MTEB benchmark. We also demonstrate how utilizing extended sequence lengths can offer our models an advantage over those without such capabilities.

References

Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 3982–3992, 2019.

Ofir Press, Noah A. Smith, and Mike Lewis. Train short, test long: Attention with linear biases enables input length extrapolation, 2022.

Michael Günther, Louis Milliken, Jonathan Geuter, Georgios Mastrapas, Bo Wang, and Han Xiao. Jina embeddings: A novel set of high-performance sentence embedding models. *arXiv preprint arXiv:2307.11224*, 2023.

Liang Wang, Nan Yang, Xiaolong Huang, Binxing Jiao, Linjun Yang, Daxin Jiang, Rangan Majumder, and Furu Wei. Text embeddings by weakly-supervised contrastive pre-training. *arXiv preprint arXiv:2212.03533*, 2022.

Scott Deerwester, Susan T. Dumais, George W. Furnas, Thomas K. Landauer, and Richard Harshman. Indexing by latent semantic analysis. *Journal of the American Society for Information Science*, 41(6):391–407, 1990.

David Blei, Andrew Ng, and Michael Jordan. Latent dirichlet allocation. In T. Dietterich, S. Becker, and Z. Ghahramani, editors, *Advances in Neural Information Processing Systems*, volume 14. MIT Press, 2001. URL https://proceedings.neurips.cc/paper_files/paper/2001/file/296472c9542ad4d4788d543508116cbc-Paper.pdf.

Tianyu Gao, Xingcheng Yao, and Danqi Chen. Simcse: Simple contrastive learning of sentence embeddings, 2022.

Luyu Gao and Jamie Callan. Condenser: a pre-training architecture for dense retrieval, 2021.

Shitao Xiao, Zheng Liu, Yingxia Shao, and Zhao Cao. Retromae: Pre-training retrieval-oriented language models via masked auto-encoder, 2022.

Zehan Li, Xin Zhang, Yanzhao Zhang, Dingkun Long, Pengjun Xie, and Meishan Zhang. Towards general text embeddings with multi-stage contrastive learning, 2023.

Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. Beir: A heterogenous benchmark for zero-shot evaluation of information retrieval models, 2021.

Niklas Muennighoff, Nouamane Tazi, Loïc Magne, and Nils Reimers. Mteb: Massive text embedding benchmark, 2023.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding, 2019.

Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. Roberta: A robustly optimized bert pretraining approach, 2019a.

Yann N. Dauphin, Angela Fan, Michael Auli, and David Grangier. Language modeling with gated convolutional networks. *CoRR*, abs/1612.08083, 2016. URL http://arxiv.org/abs/1612.08083.

Noam Shazeer. GLU variants improve transformer. *CoRR*, abs/2002.05202, 2020. URL https://arxiv.org/abs/2002.05202.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. *CoRR*, abs/1706.03762, 2017. URL http://arxiv.org/abs/1706.03762.

Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism. *CoRR*, abs/1909.08053, 2019. URL http://arxiv.org/abs/1909.08053.

Toan Q. Nguyen and Julian Salazar. Transformers without tears: Improving the normalization of self-attention. *CoRR*, abs/1910.05895, 2019. URL http://arxiv.org/abs/1910.05895.

Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *The Journal of Machine Learning Research*, 21(1): 5485–5551, 2020.

Ilya Loshchilov and Frank Hutter. Fixing weight decay regularization in adam. *CoRR*, abs/1711.05101, 2017. URL http://arxiv.org/abs/1711.05101.

Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, et al. Mixed precision training. In *International Conference on Learning Representations*, 2018.

Jeff Rasley, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. Deepspeed: System optimizations enable training deep learning models with over 100 billion parameters. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 3505–3506, 2020.

Zhuyun Dai, Vincent Y Zhao, Ji Ma, Yi Luan, Jianmo Ni, Jing Lu, Anton Bakalov, Kelvin Guu, Keith Hall, and Ming-Wei Chang. Promptagator: Few-shot dense retrieval from 8 examples. In *The Eleventh International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=gmL46YMpu2J.

Aäron van den Oord, Yazhe Li, and Oriol Vinyals. Representation learning with contrastive predictive coding. *CoRR*, abs/1807.03748, 2018. URL http://arxiv.org/abs/1807.03748.

Feng Wang and Huaping Liu. Understanding the behaviour of contrastive loss. In *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2495–2504. IEEE, 2021.

Payal Bajaj, Daniel Campos, Nick Craswell, Li Deng, Jianfeng Gao, Xiaodong Liu, Rangan Majumder, Andrew McNamara, Bhaskar Mitra, Tri Nguyen, et al. Ms marco: A human generated machine reading comprehension dataset. *arXiv preprint arXiv:1611.09268*, 2016.

Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Matthew Kelcey, Jacob Devlin, Kenton Lee, Kristina N. Toutanova, Llion Jones, Ming-Wei Chang, Andrew Dai, Jakob Uszkoreit, Quoc Le, and Slav Petrov. Natural questions: a benchmark for question answering research. *Transactions of the Association of Computational Linguistics*, 2019.

Samuel Bowman, Gabor Angeli, Christopher Potts, and Christopher D Manning. A large annotated corpus for learning natural language inference. In *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing*, pages 632–642, 2015.

Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. Training deep nets with sublinear memory cost. *arXiv preprint arXiv:1604.06174*, 2016.

Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. Roberta: A robustly optimized BERT pretraining approach. *CoRR*, abs/1907.11692, 2019b. URL http://arxiv.org/abs/1907.11692.

Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. GLUE: A multi-task benchmark and analysis platform for natural language understanding. *CoRR*, abs/1804.07461, 2018. URL http://arxiv.org/abs/1804.07461.

Jason Phang, Thibault Févry, and Samuel R. Bowman. Sentence encoders on stilts: Supplementary training on intermediate labeled-data tasks. *CoRR*, abs/1811.01088, 2018. URL http://arxiv.org/abs/1811.01088.

Eva Sharma, Chen Li, and Lu Wang. BIGPATENT: A large-scale dataset for abstractive and coherent summarization. *CoRR*, abs/1906.03741, 2019. URL http://arxiv.org/abs/1906.03741.

Wikimedia Foundation. Wikimedia downloads, 2022. URL https://dumps.wikimedia.org.

David Wadden, Shanchuan Lin, Kyle Lo, Lucy Lu Wang, Madeleine van Zuylen, Arman Cohan, and Hannaneh Hajishirzi. Fact or fiction: Verifying scientific claims. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 7534–7550, 2020.

A Appendix: MTEB and LoCo Becnharmk

Task	Accuracy [%] jina-small-v2	jina-base-v2
AmazonCounterfactualClassification	71.36	74.73
AmazonPolarityClassification	82.90	88.54
AmazonReviewsClassification	40.89	45.26
Banking77Classification	78.25	84.01
EmotionClassification	44.01	48.77
ImdbClassification	73.64	79.44
MassiveIntentClassification	67.61	71.93
MassiveScenarioClassification	69.75	74.49
MTOPDomainClassification	93.96	95.68
MTOPIntentClassification	72.50	83.15
ToxicConversationsClassification	71.54	73.35
TweetSentimentExtractionClassification	59.40	62.06
Avg	68.82	73.45

Table 4: Detailed Performance on the MTEB Classification Tasks

Task	$\mathcal{V}$ measure jina-small-v2	jina-base-v2
ArxivClusteringP2P	44.02	45.39
ArxivClusteringS2S	35.16	36.68
BiorxivClusteringP2P	35.57	37.05
BiorxivClusteringS2S	29.07	30.16
MedrxivClusteringP2P	31.86	32.41
MedrxivClusteringS2S	27.51	28.09
RedditClustering	49.28	53.05
RedditClusteringP2P	57.09	60.31
StackExchangeClustering	55.35	58.52
StackExchangeClusteringP2P	34.42	34.96
TwentyNewsgroupsClustering	41.57	42.47
Avg	40.08	41.73

Table 5: Detailed Performance on the MTEB Clustering Tasks

Task	Spearman correlation based on cos similarity jina-small-v2	jina-base-v2
SummEval	30.56	31.60

Table 6: Detailed Performance on the MTEB Summarization Tasks

Task	cos-sim-ap jina-small-v2	jina-base-v2
SprintDuplicateQuestions	95.12	95.30
TwitterSemEval2015	72.15	74.74
TwitterURLCorpus	86.05	86.09
Avg	84.44	85.38

Table 7: Detailed Performance on the MTEB Pair Classification Tasks

Task	mAP@10 jina-small-v2	jina-base-v2
AskUbuntuDupQuestions	59.62	62.25
MindSmallReranking	30.99	30.54
SciDocsRR	79.76	83.10
StackOverflowDupQuestions	49.99	52.05
Avg	55.09	56.98

Table 8: Detailed Performance on the MTEB ReRanking Tasks

Task	nDCG@10 jina-small-v2	jina-base-v2
ArguAna	46.73	44.18
ClimateFEVER	20.05	23.53
CQADupstackRetrieval	38.03	39.34
DBPedia	32.65	35.05
FEVER	68.02	72.33
FiQA2018	33.43	41.58
HotpotQA	56.48	61.38
MSMARCO	37.28	40.92
NFCorpus	30.40	32.45
NQ	51.59	60.44
QuoraRetrieval	87.19	88.20
SCIDOCS	18.61	19.86
SciFact	63.89	66.68
Touche2020	23.52	26.24
TRECCOVID	65.18	65.91
Avg	45.14	47.87

Table 9: Detailed Performance on the MTEB Retrieval Tasks

Task	Spearman correlation based on cosine similarity jina-small-v2	jina-base-v2
BIOSSES	80.52	81.23
SICK-R	76.72	79.65
STS12	73.66	74.27
STS13	83.30	84.18
STS14	79.17	78.81
STS15	87.30	87.55
STS16	83.61	85.35
STS17(en-en)	88.23	88.88
STS22(en)	63.46	62.20
STSBenchmark	84.04	84.84
Avg	80.00	80.70

Table 10: Detailed Performance on the MTEB STS Tasks

Model	Fine-tuned on LoCo	Parameters	Context Length	avg. nDCG@10
M2-BERT-32768	✓	80M	32,768	92.5
e5-mistral-7b-instruct		7.3B	4,096	88.5
M2-BERT-32768	✓	80M	8,192	85.9
jina-base-v2		137M	8192	85.4
bge-large-en-v1.5	✓	335M	512	85.0
M2-BERT-2048	✓	80M	2,048	83.6
jina-small-v2		33M	8,192	83.4
bge-base-en-v1.5	✓	109M	512	83.0
bge-small-en-v1.5	✓	33M	512	81.2
bge-large-en-v1.5		335M	512	77.2
bge-base-en-v1.5		109M	512	73.4
bge-small-en-v1.5		33M	512	70.6
cohere-embed-v3		NA	512	66.6
ada-embeddings-002		NA	8,191	52.7
voyage-v1		NA	4,096	25.4

Table 11: Performance on the new LoCo Dataset